

Graduate Algorithms

Problems on Divide and Conquer - Answers

04 September 2026

What's rewarding about making stuff is figuring it out on your own or with a group of people and getting to share in that feeling.

I understand how tempting a shortcut can be, especially when you go online and you see all these other people doing cool things and you're like, 'man, I'll never be that talented. It would take me like 10 years to learn how to make music like that.'

Okay, that's fine. What's the rush? Just start right now. Don't worry about how long it's going to take. Just take the first step. How are you ever going to do something for 10 years if you won't even do it for one day?

Maybe you won't be good at something the first time you do it. You probably won't. But every time you improve, that's something that you can feel proud of. That's something that you did.

— Drew Gooden

☰ Counting Inversions

Given an (1-indexed) array A of n distinct integers, Give an algorithm to count the number of inversions in $O(n \log n)$

An inversion is a pair of i, j such that

- $1 \leq i < j \leq n$
- $A[i] > A[j]$

Hint: Consider a function `def inversions(A: list[int]) -> tuple[int, list[int]]` where we are returning both the number of inversions and sorted list A .

Let there be two sorted X and Y such that $X \cup Y = A$. What can we say about $\text{inversions}(A)$?

As X and Y are sorted, there are no inversions in X or Y . We now need to consider the inversions between X and Y . We could do so trivially in $|X| \cdot |Y|$ time. However, notice, if for some i, j ; $X[i] > Y[j]$ then we can simply add $|X| - i$ inversions as $X[i + 1] > Y[j], X[i + 2] > Y[j], \dots$

This means, we could run a merge procedure (similar to merge sort) on X, Y and make a count of inversions on the way. This would take $O(|X| + |Y|) = O(|A|) = O(n)$ time.

Using the above idea, we could simply split our array into two equal parts X and Y ; count the number of inversions in each of these parts while sorting it and then merging them to count the inversion between them. The pseudocode would look like:

INVERSIONS

```
1 def InverMerge(X, len_x, Y, len_y):
```

INVERSIONS

```

2   |   if |X| = 0:
3   |       |   return (0, Y)
4   |   if |Y| = 0
5   |       |   return (0, X)
6   |   if X[1] >= Y[1]:
7   |       |   (inv, lis) = InverMerge(X[2-n], len_x-1, Y, len_y)
8   |       |   return (inv, X[1] ++ lis)
9   |   if X[1] < Y[1]:
10  |       |   (inv, lis) = InverMerge(X, len_x, Y[2-n], len_y)
11  |       |   return (inv + len_x, Y[1] ++ lis)
12  |   def Inversions(A):
13  |       |   n = len(A)
14  |       |   X = A[1 - ⌊ $\frac{n}{2}$ ⌋]
15  |       |   Y = A[(⌊ $\frac{n}{2}$ ⌋ + 1) - n]
16  |       |   (inv_X, lis_X) = Inversions(X)
17  |       |   (inv_Y, lis_Y) = Inversions(Y)
18  |       |   (inv, lis) = InverMerge(lis_X, ⌊ $\frac{n}{2}$ ⌋, lis_Y, ⌈ $\frac{n}{2}$ ⌉)
19  |       |   return (inv_X + inv_Y + inv, lis)

```

We can see that the time complexity is $T(n) = 2T(\frac{n}{2}) + O(n) \Rightarrow T(n) = O(n \log n)$ as desired.

☰ Strassen's Algorithm

Given two $n \times n$ matrices A and B , compute their product AB in time $O(n^{\log_2 7})$

Assuming $m \geq n$, can you find an $O(m^2 n^{0.81})$ algorithm to multiply A a $m \times n$ and B a $n \times m$ matrix using the above a subroutine. (Hint: Multiply $\lceil \frac{m}{n} \rceil^2$ pairs of order- n matrices)

Assuming $m \geq n \geq t$, Can you find an $O(mnt^{0.81})$ algorithm to multiply A a $m \times n$ and B a $n \times t$ matrix using the above a subroutine. (Hint: Multiply pairs of $t \times t$ matrices)

The idea for the first part is that $O(n^{\log_2 7})$ is perhaps the result of the recurrence $T(n) = 7T(\frac{n}{2}) + \Omega(n^{\log_2 7})$.

Note, the Ω at the end. Any merge step taking time less than $n^{\log_2 7}$ is enough.

Thus, we consider splitting the $n \times n$ matrix into $4 \frac{n}{2} \times \frac{n}{2}$ matrices. Let

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

$$\Rightarrow A \cdot B = \begin{pmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{pmatrix}$$

Thus, we have a way to solve our multiplication by solving 8 products. Sadly, $T(n) = 8T(\frac{n}{2}) + O(n^2) \Rightarrow T(n) = O(n^{\log_2 8}) = O(n^3)$.

So we want to reduce the number of multiplications to 7. As it turns out, if you stare at this long enough, you can do that. Check out <https://cs.stackexchange.com/a/130028> for the intuition. This is named Strassen's Algorithm.

For the next part, Since $m \geq n$, we can view the tall matrix A as a vertical stack of $n \times n$ blocks and the wide matrix B as a horizontal row of $n \times n$ blocks (upto padding with 0s). Let $q = \lceil \frac{m}{n} \rceil$.

Basically,

$$A = \begin{pmatrix} A_1 \\ A_2 \\ \dots \\ A_q \end{pmatrix}, B = (B_1 \ B_2 \ \dots \ B_q)$$

where each A_i and B_j is an $n \times n$ block. The resulting product $C = AB$ is an mm matrix consisting of q^2 blocks, where each block C_{ij} is defined by:

$$C_{ij} = A_i \cdot B_j$$

Since each A_i and B_j is $n \times n$, we can compute each C_{ij} using the Strassen's Algorithm in $O(n^{2.81})$.

We perform q^2 multiplications. Since $q \approx \frac{m}{n}$, the total time is:

$$O\left(\left(\frac{m}{n}\right)^2 n^{2.81}\right) = O(m^2 n^{0.81})$$

For last part, Now we have $m \geq n \geq t$. The goal is to reach $O(mnt^{0.81})$. Following the logic from Part (a), we should partition everything into $t \times t$ blocks (upto padding by 0s).

1. Partition A ($m \times n$) into a grid of blocks $A_{i,j}$ where $1 \leq i \leq \frac{m}{t}$ and $1 \leq j \leq \frac{n}{t}$.
2. Partition B ($n \times t$) into a vertical stack of blocks $B_{j,1}$ where $1 \leq j \leq \frac{n}{t}$.
3. The resulting matrix C ($m \times t$) will have $\frac{m}{t}$ blocks.

Each block $C_{i,1}$ of the output is computed as:

$$C_{i,1} = \sum_{j=1}^{\frac{n}{t}} A_{i,j} \cdot B_{j,1}$$

To compute one block $C_{i,1}$, we perform $\frac{n}{t}$ multiplications of $t \times t$ matrices. There are $\frac{m}{t}$ such blocks in C . Hence, we made a total of $\frac{m}{t} \cdot \frac{n}{t} = \frac{mn}{t^2}$ multiplications. Each multiplication takes $O(t^{2.81})$.

Hence, the total time complexity is $O(mnt^{0.81})$.

☰ Merging Arrays

Let $A_1, A_2, \dots, A_k$ be k arrays, each of which has been sorted.

These arrays are mutually disjoint, namely, no integer can appear in more than one array.

Design an algorithm to merge the k arrays into one sorted array in $O(n \log k)$ time, where n is the total length of the k arrays.

Note: these arrays may have different lengths.

The idea is rather boring and better elucidated by pseudocode.

MERGING ARRAYS

```

1  def Merge(X, Y):
2      if |X| = 0
3          |   return Y
4      if |Y| = 0
5          |   return X
6      if X[1] >= Y[1]:
7          |   lis = Merge(X[2-n], Y)
8          |   return X[1] ++ lis
9      if X[1] < Y[1]:
10         |   lis = Merge(X, Y[2-n])
11         |   return Y[1] ++ lis
12  def Merge_Arrays(A : List of Lists -> List):
13      k = len(A)
14      if k = 0:
15          |   return []
16      if k = 1
17          |   return A[1]
18      A_left = Merge_Arrays(A[1 - ⌊ $\frac{k}{2}$ ⌋])
19      A_right = Merge_Arrays(A[(⌊ $\frac{k}{2}$ ⌋ + 1) - k])
20      return Merge(A_left, A_right)

```

Notice, the recurrence here is a bit more interesting.

$$T(n, k) = T\left(\sum_{i=1}^{\lfloor \frac{k}{2} \rfloor} A_i, \left\lfloor \frac{k}{2} \right\rfloor\right) + T\left(\sum_{i=\lfloor \frac{k}{2} \rfloor + 1}^k A_i, \left\lceil \frac{k}{2} \right\rceil\right) + O(n)$$

This seems completely impenetrable. So what do we do?

Notice that at every step, every element of the lists is moved atmost twice (once in the Merge_Arrays and once in Merge). This implies, we could sort of get rid of the n from the recurrence. This could allow us to say:

$$T(k) = T\left(\left\lfloor \frac{k}{2} \right\rfloor\right) + T\left(\left\lceil \frac{k}{2} \right\rceil\right) + O(n)$$

which is the same recurrence as MergeSort and admits a similar answer: the recurrence tree can go atmost down till $\log k$ levels. The nodes at each level sum upto n . Thus, $T(k) = O(n \log k)$.

☰ Max Weight Subarray

Let A be an array of n integers (A is not necessarily sorted). Each integer in A may be positive or negative.

Given i, j satisfying $1 \leq i \leq j \leq n$,

Define sub-array $A[i : j]$ as the sequence $(A[i], A[i + 1], \dots, A[j])$, and the weight of $A[i : j]$ as $A[i] + A[i + 1] + \dots + A[j]$.

For example, consider $A = (13, -3, -25, 20, -3, -16, -23, 18)$; $A[1 : 4]$ has weight 5, while $A[2 : 4]$ has weight -8 .

1. Give an algorithm to find a sub-array of with the largest weight, among all sub-arrays $A[i : j]$ with $j = n$. Your algorithm must finish in $O(n)$ time.
2. Give an algorithm to find a sub-array with the largest weight in $O(n \log n)$ time (among all the possible sub-arrays).

For part 1, Let $s_i = a_i + a_{i+1} + \dots + a_n$. Notice, $s_{i+1} = s_i - a_i$. Using this, we can compute s_1 to s_n in $O(n)$ time and then find the maximum element.

MAX WEIGHT SUBARRAY I (SUFFIX)

```

1  def solveI_suffix(A):
2      n = len(A)
3      suf ← empty array of length n
4      suf[n] = A[n]
5      for i in (n-1, 1, -1):
6          └   suf[i] = suf[i+1] - a[i+1]
7      └   return max(suf)
```

We could also do this by computing $p_i = a_1 + a_2 + \dots + a_i$ and noticing $s_i = \text{sum}(A) - p_{i-1} = p_n - p_{i-1}$ and $p_{i+1} = p_i + a_{i+1}$.

MAX WEIGHT SUBARRAY I (PREFIX)

```

1  def solveI_prefix(A):
2      n = len(A)
3      pre ← empty array of length n
4      pre[1] = A[1]
5      for i in (1, n-1):
6          └   pre[i] = pre[i-1] + a[i]
7      └   return (pre[n] - min(pre))
```

Both of these are clearly $O(n)$ as it takes $O(n)$ time to make the prefix/suffix arrays and then $O(n)$ time to get the minimum/maximum element.

For part 2, consider splitting A into two arrays L and R (such that $L ++ R = A$). The maximum weight sub-array is either completely in L , completely in R or between them.

If it is between them, then has to be the sum of the max weight suffix of L and max weight prefix of R . This gives us the following pseudocode

MAX WEIGHT SUBARRAY II

```

1  def MaxWeightSubarray(A):
2      n = len(A)
3      L = A[1 - ⌊ $\frac{n}{2}$ ⌋]
4      R = A[(⌊ $\frac{n}{2}$ ⌋ + 1) - n]
5      max_L = MaxWeightSubarray(L)
6      max_R = MaxWeightSubarray(R)
7      pre ← empty array of length ⌊ $\frac{n}{2}$ ⌋
8      pre[1] = A[1]
9      for i in (1, ⌊ $\frac{n}{2}$ ⌋):
10         ⌊ pre[i] = pre[i-1] + a[i]
11     suf ← empty array of length ⌈ $\frac{n}{2}$ ⌉
12     suf[n] = A[n]
13     for i in (⌈ $\frac{n}{2}$ ⌉ - 1, 1, -1):
14         ⌊ suf[i] = suf[i+1] - a[i+1]
15     max_LR = max(pre) + max(suf)
16     ⌊ return max(max_L, max_R, max_LR)

```

Notice, the time completely is

$$\begin{aligned}
 T(n) &= T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + O\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + O\left(\left\lceil \frac{n}{2} \right\rceil\right) \\
 &= T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + O(n)
 \end{aligned}$$

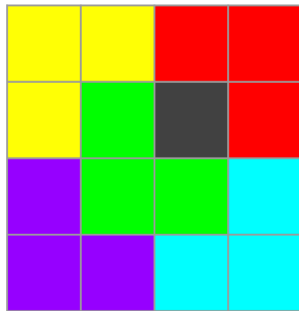
which is same as MergeSort and thus, $T(n) = O(n \log n)$.

☰ L Tiles

Given a grid of $2^N \times 2^N$, all cells are initially empty except one of the cells. Let the blocked cell be B .

Write an algorithm to figure out a way to fill the grid with L shapes (without overlapping other tiles or B) or report “NO” if that is not possible. Your algorithm should run in $O(2^N)$.

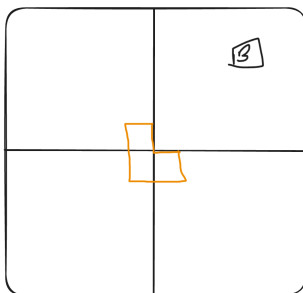
The $N = 2, B = (3, 2)$ case is given below.



Hint : We never have to return “NO”.

This is just the algorithms version of a classic induction question.

The idea is to divide the $2^N \times 2^N$ square into 4 $2^{N-1} \times 2^{N-1}$ squares. One of these has a hole and is a smaller instance of our original problem. For the other 3, we put the first L to cover a square each. The diagram might make it more clear.



Thus, we can induct on the smaller squares to show that a tiling always exists (formalizing the proof is left upto reader).

Finally, notice that for 3 of the instances, the tilings are isomorphic. Thus, $T(n) = 2T(n-1) + O\left(\frac{2^{2n-2}-1}{3}\right) = 2T(n-1) + O(4^n) \Rightarrow T(n) = O(4^n)$ as desired.

☰ Big Mod

Given p, e, b , compute $p^e \bmod b$ in $O(\log e)$ time.

The idea is that $p^e = (p^{\frac{e}{2}})^2$. The algorithm is simple:

 BIG MOD

```

1  def modExp(p,e,b):
2      if e = 0:
3          └─ return 1
4      if e = 1:
5          └─ return p mod b
6      if e mod 2 == 0:
7          └─ return (modExp(p, e/2, b))2 mod b
8      else:
9          └─ return p * modExp(p, e-1, b) mod b
  
```

The complexity is $T(e) = T(\lfloor \frac{e}{2} \rfloor) + O(1) \Rightarrow T(e) = O(\log e)$ as desired.

i Remark

We technically have ignored the time taken to multiply two n bit integers. Let $M(n)$ be the time to multiply two n bit integers.

As the multiplications are modulo b , n is bounded by $\log(b)$. Thus, we would have $O(\log(e)M(\log(b)))$.

The best known bounds for $M(n) = O(n \log n)$ (Harvey, van der Hoeven 2019) while we can achieve $M(n) = O(n \log n \log \log n)$ using stuff we have seen in the course.

☰ DeadShot (DS Algo Mock-Midsem 2026)

To celebrate the end of midsems, a group of students head to the CLAB to play Deadshot.io. There are n players participating, and the i th player has a skill score of $A[i]$.

They want to split into two teams for a match. They agree to the following rules:

- Team Kabir must have a total skill score greater than or equal to Team Piyush.
- The difference between the total skill scores of the two teams should be as small as possible.

Write a function `balanced_teams(A)` that returns the minimum possible difference between the total skill scores of Team Kabir and Team Piyush under these rules in $O(n2^{\frac{n}{2}})$ time.

Input	Output	Explanation
$A = [5, 4, 7, 6]$	0	Team Kabir gets $5 + 6 = 11$ and Team Piyush gets $4 + 7 = 11$ skill.
$A = [10, 1, 1]$	8	Team Kabir gets 10 skill Team Piyush gets $1 + 1 = 2$ skill.
$A = [3, 4, 2, 2]$	1	Team Kabir gets $4 + 2 = 6$ skill and Team Piyush gets $3 + 2 = 5$ skill.

Hint: The intended solution uses $O(2^{\frac{n}{2}})$ space as we need **two** arrays of that length...(This should ring some bells)

- Write pseudocode implementing `balanced_teams(A)`. You are not allowed to import libraries or use any language specific features.
- Argue that your algorithm runs in $O(n2^{\frac{n}{2}})$.
- Argue the correctness of your implementation.

There is an obvious brute force $O(2^n)$ algorithm where we consider each possible partition. How do we make it faster?

Consider splitting the list into two halves L and R . Consider the partitions of L and partitions of R , given a partition in L , we want to find it's best mate in R .

This would trivially take $O(2^{\frac{n}{2}} * 2^{\frac{n}{2}}) = O(2^n)$ time. However, we could do better if we sort the partitions of L and R according to $T_k - T_p$. We can now find an optimal set as follows:

BALANCED TEAMS I

```

1  def balanced_teams(A):
2      n ← length(A)
3      m ← floor(n / 2)
4      L ← all signed sums obtainable from A[1..m]
5      R ← all signed sums obtainable from A[m+1..n]
6      sort(L) in increasing order
7      sort(R) in increasing order
8      i ← 1
9      j ← length(R)
10     ans ← infinity
```

BALANCED TEAMS I

```

11   while i <= length(L) and j >= 1:
12       while j > 1 and L[i] + R[j - 1] >= 0:
13           j ← j - 1
14       if L[i] + R[j] >= 0:
15           ans ← min(ans, L[i] + R[j])
16       i ← i + 1
17   return ans

```

The cost is dominated by sorting and hence, the time complexity is $O(2^{\frac{n}{2}} \log(2^{\frac{n}{2}})) = O(\frac{n}{2} 2^{\frac{n}{2}}) = O(n 2^{\frac{n}{2}})$ as desired.